

Java Thread:

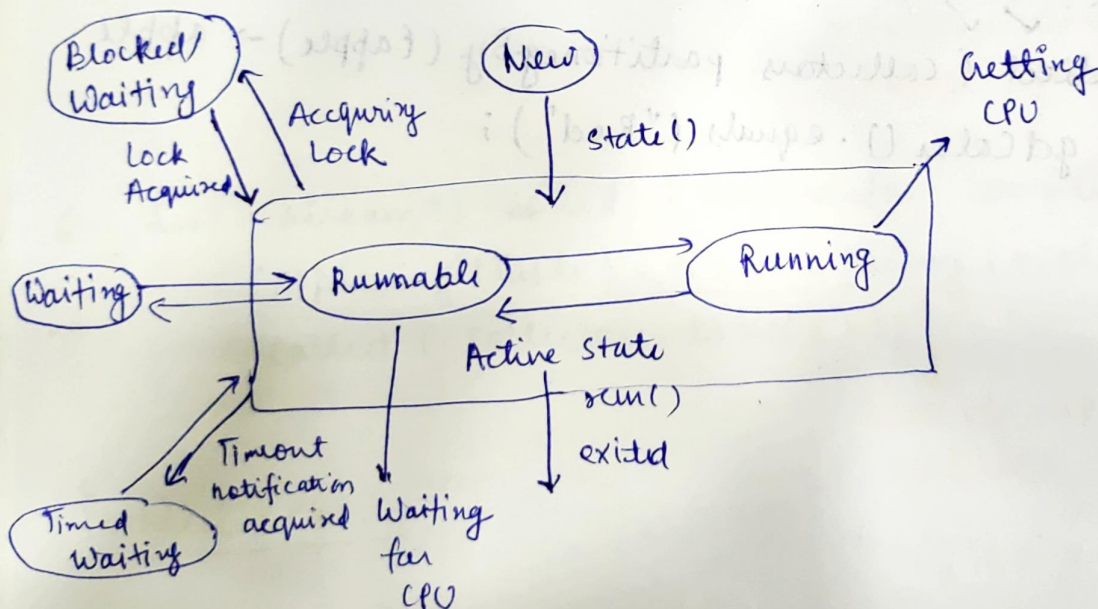
Concurrent — Two threads → Both at a time

Simultaneous — Two threads → One at a time

ka preparing
Abusing boy
TV,
Vegetable
cut

- Thread is flow of execution.
- Thread are within the process. (same address space)
- Create thread

- Extends threads
- implement runnable
- implement callable
- Using java util. Concurrent. Executor frame work



Thread blocked state -

- Sleeping -
- Join - Waiting for other threads to finish.
-
-

run() → while execution.
start() → start thread.
stop() → stop thread.

*
*
*
① When u r extending thread, override run also.
② obj.start() → create a thread & start it separately & main will continue to execute.

③ Thread sequence is not controlled.

④ thread 1.join() → thread 1 will wait for all the other threads to finish

POJO
⑤ All classes are POJO (Plain old java object) except static.

⑥ POJI - Plain old java interface

⑦ Run method will not work until we start it.
thread.start(); → without this it will act as simple object.

thread.sleep(x); → not recommended to use
↓
millisecond
↓
Because you are asking thread to sleep → which will take time.

⑧ Thread.currentThread().getName() - given name of current thread.

??

@ Functional Interface

Runnable interface :

→ `run()` Takes Nothing
Gives Nothing

1. Using implementing class
2. Anonymous inner class
3. Lambda

① Runnable does not have start method.

② We need to give object of runnable interface to thread.

→ `NumberPrinterRunnable task1 = new NumberPrinterRunnable();`

→ 1. `Thread t1 = new Thread(task1);`

2. `Thread t1 = new Thread(new Runnable() {
run();
});`

3. Lambda

@ Functional Interface

Callable interface : `<V> ??`

↓
`call()` Takes Nothing
will give

① Thread do not have callable constructor.

② Callable is used for future task.

↓
implements runnable function

```
FutureTask<Integer>[] sumTask = new FutureTask[5];
```

```
for (i → 5)
```

```
{  
    sumTask[i] = new FutureTask<Integer>  
        (callable);
```

Thread Safe Collection — Copy On Write ArrayList <>();
Concurrent Hash Map <>();, vector

Thread Iterator = new Thread() → { list.forEach (System.
out :: println); } , "iteratorName");

Thread (Runnable, String)

↓
name

Interview

= Q. How to make arraylist thread safe?

```
class UnsafeCollection {
```

```
    static void main (String[] args) {
```

```
        List<Integer> list = new ArrayList<>();
```

```
        list = Collections.synchronizedList(list);
```

↳ synchronized list

```
    } Map<String, String> map = new HashMap<>();
```

```
        map = Collections.synchronizedMap(map);
```

```
}
```

wait () → wait for indefinite time

wait (t) → wait for definite time t.

```
public synchronized void printEven (int n);
```

```
public synchronized void printOdd (int n);
```

used because it is shared

notify() - notify the to the first waiter.

notifyAll() - notify all to the waiters.

Object → Text File → File Print Writer

Object → Bin File → Object Output Stream

Executor Framework:

ExecutorService service = Executors.newFixedThreadPool(3);
↓
create pool of 3 thread

service.execute() → sysout("Hi");

execute() → No return

submit() → return

Synchronized → Queue

① When method is static & synchronized, lock is achieved on class.

② When you synchronize the instance method, lock is achieved on object.

→ instance + instance → parallel

synchronized + sync → sequential

static sync + instance sync → parallel

static sync + static sync → sequential

Consumer - Producer Problem -

BlockingQueue < Message > syncQues = new ArrayBlockingQueue<>(100);

Update Anamoly - Multiple people tried to update same value.

Thread Synchronization - oddEvenTester

1. Cloneable Interface → clone() → deep cloning in Java.

2. Inner Class

3. Optional (Java 8 feature)

4. Reflection